

Steady-State Error Analysis

ME 2801 — Week 6

Contents

1	Steady-State Error Analysis	2
1.1	Introduction	2
1.2	Deriving the General Error Formula	2
1.3	Test Inputs: Step, Ramp, Parabola	3
1.3.1	Defense Analogies	3
1.4	Step, Ramp, and Parabola Errors from Eq. (1.1)	4
1.4.1	Step Input	4
1.4.2	Ramp Input	5
1.4.3	Parabola Input	5
1.5	System Type	5
1.6	Static Error Constants	6
1.6.1	Steady State Error Specification	7
1.6.2	The Summary Table Falls Out of One Formula	7
1.7	Design: Choosing Gain to Meet an Error Specification	10
1.7.1	Procedure	10
1.7.2	Example	10
1.8	Eliminating Steady-State Error with an Integrator	10
1.8.1	The Adaptation Argument	11
1.8.2	Effect on System Type	11
1.8.3	The Trade-offs	12
1.9	Integrator vs. Feedforward: Practical Trade-offs	12
1.9.1	Feedforward Does Not Change System Type	12
1.9.2	Side-by-Side Comparison	14
1.9.3	The Combined Approach: PI with Feedforward	14

Chapter 1

Steady-State Error Analysis

1.1 Introduction

The PID article established how a feedback controller shapes the *transient* response — settling time, overshoot, damping ratio. But there is a second performance dimension that transient analysis says nothing about: what happens *after* the transients die out? If a vessel is commanded to hold 5 m/s, does it actually reach 5 m/s, or does it settle 0.3 m/s short? If it is commanded to track a linearly increasing waypoint sequence, does the position error grow without bound, or does it converge to a fixed lag (constant offset)?

Steady-state error is the difference between the desired output and the actual output after transients have decayed. It is the next essential tool for describing and specifying the performance of a feedback system.

In this article we derive steady-state error expressions for *unity negative feedback* systems (Fig. 1.1) with *stable* closed-loop dynamics. The final value theorem, on which all steady state error analysis rests, is only valid for stable systems.

1.2 Deriving the General Error Formula

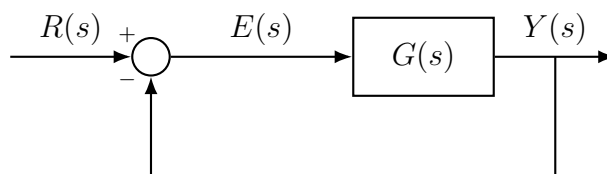


Figure 1.1: Unity-feedback loop. $G(s)$ is the open-loop forward transfer function (plant $\times$ controller combined). The error signal is $E(s) = R(s) - Y(s)$.

For the loop in Fig. 1.1, the plant output is $Y(s) = G(s)E(s)$. Substituting into the error

definition $E(s) = R(s) - Y(s)$:

$$E(s) = R(s) - G(s)E(s) \implies E(s) = \frac{R(s)}{1 + G(s)}$$

We want the long-run value of $e(t)$. The **final value theorem** avoids a full inverse Laplace transform:

$$e(\infty) = \lim_{t \rightarrow \infty} e(t) = \lim_{s \rightarrow 0} s E(s)$$

This is valid as long as $sE(s)$ has no poles on or to the right of the imaginary axis — which holds whenever the closed-loop system is stable. Substituting:

$$\boxed{e(\infty) = \lim_{s \rightarrow 0} \frac{s R(s)}{1 + G(s)}} \quad (1.1)$$

This is the master equation. Everything that follows is obtained by substituting specific inputs $R(s)$ and specific open-loop transfer functions $G(s)$. No new derivation is needed; only the evaluation of limits.

Notation. Throughout this article, $G(s) = C(s)P(s)$ denotes the *combined* open-loop forward transfer function — controller times plant. This is *overloaded notation*: the symbol $G(s)$ sometimes refers to the plant alone, sometimes to the controller, and sometimes to their product, depending on context. In steady-state error analysis the convention is always $G(s) = C(s)P(s)$: the full forward path from the error signal to the plant output. The formula depends only on that product, not on how it is divided between $C(s)$ and $P(s)$.

1.3 Test Inputs: Step, Ramp, Parabola

The choice of test input depends on what the controlled system must track. We typically test with three standard inputs, which can be thought of as three different scenarios:

- **Step.** A step reference represents a sudden change in desired position or speed that must then be held. A vessel commanded to a new waypoint and then to hold position faces a step problem.
- **Ramp.** A ramp reference represents a target moving at constant velocity. The tracking system must continuously follow a linearly increasing reference.
- **Parabola.** A parabola reference represents a target with constant acceleration. The reference grows quadratically, which is the hardest case to follow.

1.3.1 Defense Analogies

These three test inputs map onto the three targeting scenarios that appear in fire control and weapon-system tracking:

Table 1.1: Standard test inputs for steady-state error analysis.

Name	Physical interpretation	$r(t)$	$R(s)$
Step	Constant position	1	$1/s$
Ramp	Constant velocity	t	$1/s^2$
Parabola	Constant acceleration	$\frac{1}{2}t^2$	$1/s^3$

Stationary target. A fixed location, a satellite in geostationary orbit, a ship at anchor — the tracking system only needs to acquire and hold a fixed pointing angle. The test input is a step. Even a small persistent pointing bias (nonzero step error) causes a miss.

Constant-velocity target. A surface ship at steady speed, a satellite in circular orbit, an aircraft in level flight. The system must continuously *follow* a moving position. A Close-In Weapon System (CIWS) director computing gun lead angles, or a satellite dish tracking a low-earth-orbit pass, faces exactly this problem. The ramp tracking error is the steady-state angular or positional lag behind the target.

Maneuvering (accelerating) target. A ballistic missile under thrust, an aircraft in a turn, a UAV executing an evasive break. The target’s velocity itself is changing. Even a system that tracks constant-velocity targets perfectly will accumulate growing error against an accelerating target unless the control system has sufficient integration in its forward path. This is the parabolic case.

The sophistication required of the feedback system increases across the three cases. As we will see, each successive case demands one additional integrator in the open-loop forward path to maintain bounded error.

This progression captures a tension that runs through all of control engineering: improving steady-state accuracy generally costs something — slower transient response, reduced stability margin, or both. Knowing which side of that tradeoff you are on is as important as knowing the formula.

1.4 Step, Ramp, and Parabola Errors from Eq. (1.1)

1.4.1 Step Input

With $R(s) = 1/s$, Eq. (1.1) gives:

$$e_{\text{step}}(\infty) = \lim_{s \rightarrow 0} \frac{s \cdot (1/s)}{1 + G(s)} = \lim_{s \rightarrow 0} \frac{1}{1 + G(s)} = \frac{1}{1 + \lim_{s \rightarrow 0} G(s)} \quad (1.2)$$

The key quantity is $\lim_{s \rightarrow 0} G(s)$, the DC gain of the open-loop transfer function.

- If $G(s)$ has *no poles* at $s = 0$: the DC gain is a finite constant, and the step error is nonzero.
- If $G(s)$ has *one or more poles* at $s = 0$ (integrators): $G(0) = \infty$, and $e_{\text{step}} = 0$.

This is exactly what we found in the PID article: a proportional-only controller leaves a residual step error, while adding an integrator (PI or I action) drives it to zero.

1.4.2 Ramp Input

With $R(s) = 1/s^2$:

$$e_{\text{ramp}}(\infty) = \lim_{s \rightarrow 0} \frac{s \cdot (1/s^2)}{1 + G(s)} = \lim_{s \rightarrow 0} \frac{1}{s(1 + G(s))} = \frac{1}{\lim_{s \rightarrow 0} s G(s)} \quad (1.3)$$

The key quantity is now $\lim_{s \rightarrow 0} s G(s)$. As $s \rightarrow 0$, the added factor of s cancels one pole at the origin:

- *No integrators in $G(s)$* : $\lim s G(s) = 0$, so $e_{\text{ramp}} = \infty$. The output falls infinitely far behind a ramp input.
- *One integrator*: write $G(s) = \tilde{G}(s)/s$, where $\tilde{G}(s)$ is $G(s)$ with the single integrator removed, so $\tilde{G}(0)$ is finite and nonzero. Then $\lim s G(s) = \tilde{G}(0)$ and e_{ramp} is finite.
- *Two or more integrators*: $\lim s G(s) = \infty$, so $e_{\text{ramp}} = 0$.

1.4.3 Parabola Input

With $R(s) = 1/s^3$:

$$e_{\text{para}}(\infty) = \lim_{s \rightarrow 0} \frac{s \cdot (1/s^3)}{1 + G(s)} = \frac{1}{\lim_{s \rightarrow 0} s^2 G(s)} \quad (1.4)$$

- *Zero or one integrators*: $\lim s^2 G(s) = 0 \Rightarrow e_{\text{para}} = \infty$.
- *Two integrators*: $\lim s^2 G(s)$ is finite and nonzero $\Rightarrow$ finite parabola error.
- *Three or more integrators*: $\lim s^2 G(s) = \infty \Rightarrow e_{\text{para}} = 0$.

The pattern is clear: each successive input type requires one more integrator in $G(s)$ to achieve finite (or zero) error.

1.5 System Type

The behavior above depends on how many poles $G(s)$ has at the origin. The number of poles at $s = 0$ (the origin) is the *system type*.

Definition. The *system type* is the number n of pure integrators in the open-loop forward transfer function:

$$G(s) = \frac{K(s + z_1)(s + z_2) \cdots}{s^n(s + p_1)(s + p_2) \cdots} \quad (1.5)$$

A system is *Type 0* when $n = 0$, *Type 1* when $n = 1$, *Type 2* when $n = 2$, and so on.

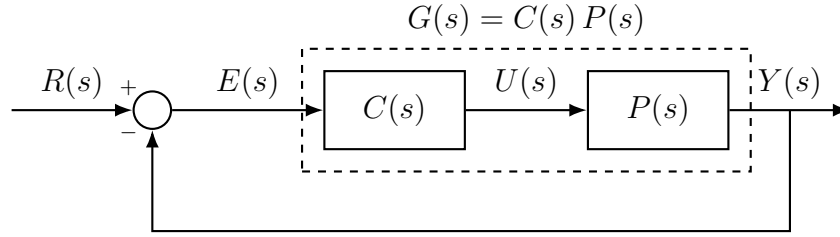


Figure 1.2: Unity-feedback loop showing controller $C(s)$ and plant $P(s)$ separately. The dashed box is the open-loop forward transfer function $G(s) = C(s)P(s)$. System type is a property of $G(s)$ — the combined product, regardless of how poles are distributed between C and P .

System type is a property of the *combined* open-loop transfer function — controller times plant, $G(s) = C(s)P(s)$ (Fig. 1.2). Adding an integrator to the controller (e.g., converting P to PI) increments the type by one.

The three limits from Section 1.4 now read directly from the type:

	$\lim_{s \rightarrow 0} G(s)$	$\lim_{s \rightarrow 0} s G(s)$	$\lim_{s \rightarrow 0} s^2 G(s)$
Type 0	K (finite)	0	0
Type 1	∞	K_v (finite)	0
Type 2	∞	∞	K_a (finite)

Higher type is not always better. Each additional integrator makes the loop harder to stabilize and more sensitive to noise and model error. Type 0 is the most common; Type 1 is used when ramp tracking is required; Type 2 is uncommon and requires careful compensator design to maintain stability.

Example. Consider the first-order plant $P(s) = 5/(s + 5)$ (Type 0). The table shows what happens as the controller changes:

Controller $C(s)$	Open-loop $G(s) = C(s)P(s)$	Type	Step error
$K_P = 2$ (P only)	$\frac{10}{s + 5}$	0	$\frac{1}{1 + 2} = \frac{1}{3}$
$K_P + K_I/s$ (PI)	$\frac{5(K_P s + K_I)}{s(s + 5)}$	1	0

The plant is identical in both rows. Adding K_I/s to the controller increments $G(s)$ from Type 0 to Type 1 and drives the step error to zero.

1.6 Static Error Constants

The three limits that appear in Eqs. (1.2) to (1.4) are given names because they appear so frequently:

$$K_x = \lim_{s \rightarrow 0} G(s) \quad (1.6)$$

$$K_v = \lim_{s \rightarrow 0} s G(s) \quad (1.7)$$

$$K_a = \lim_{s \rightarrow 0} s^2 G(s) \quad (1.8)$$

These are the *static error constants* (position, velocity, and acceleration constants, respectively).

(Many textbooks write the position constant as K_p . We use K_x throughout to avoid confusion with the proportional gain K_P of a PID controller, which appears in the same design calculations.)

With this shorthand, the errors become:

$$e_{\text{step}}(\infty) = \frac{1}{1 + K_x} \quad (1.9)$$

$$e_{\text{ramp}}(\infty) = \frac{1}{K_v} \quad (1.10)$$

$$e_{\text{para}}(\infty) = \frac{1}{K_a} \quad (1.11)$$

These are identical to Eqs. (1.2) to (1.4) — the constants are simply names for the limits already evaluated.

1.6.1 Steady State Error Specification

A single error constant encodes system type, test input, and error magnitude simultaneously. Given the specification $K_v = 20$, you can immediately conclude:

1. The system is *stable* (otherwise the spec is undefined).
2. The system is *Type 1* — only Type 1 has a finite nonzero K_v (Type 0 gives $K_v = 0$; Type 2 gives $K_v = \infty$).
3. The relevant test input is a *ramp*.
4. The steady-state ramp error per unit slope is $1/K_v = 0.05 = 5\%$.

1.6.2 The Summary Table Falls Out of One Formula

Table 1.2 collects every case. Every entry follows directly from Eq. (1.1) — no new derivations or memorization required. The diagonal pattern is the key: Type n achieves zero error for the step and all simpler inputs, finite error for input type n (the one that matches the number of integrators), and infinite error for harder inputs.

Table 1.2: Steady-state errors as a function of system type and input. Every entry follows from Eq. (1.1). ∞ indicates the output diverges from the reference.

Input	System Type		
	Type 0	Type 1	Type 2
Step	$\frac{1}{1 + K_x}$	0	0
Ramp	∞	$\frac{1}{K_v}$	0
Parabola	∞	∞	$\frac{1}{K_a}$

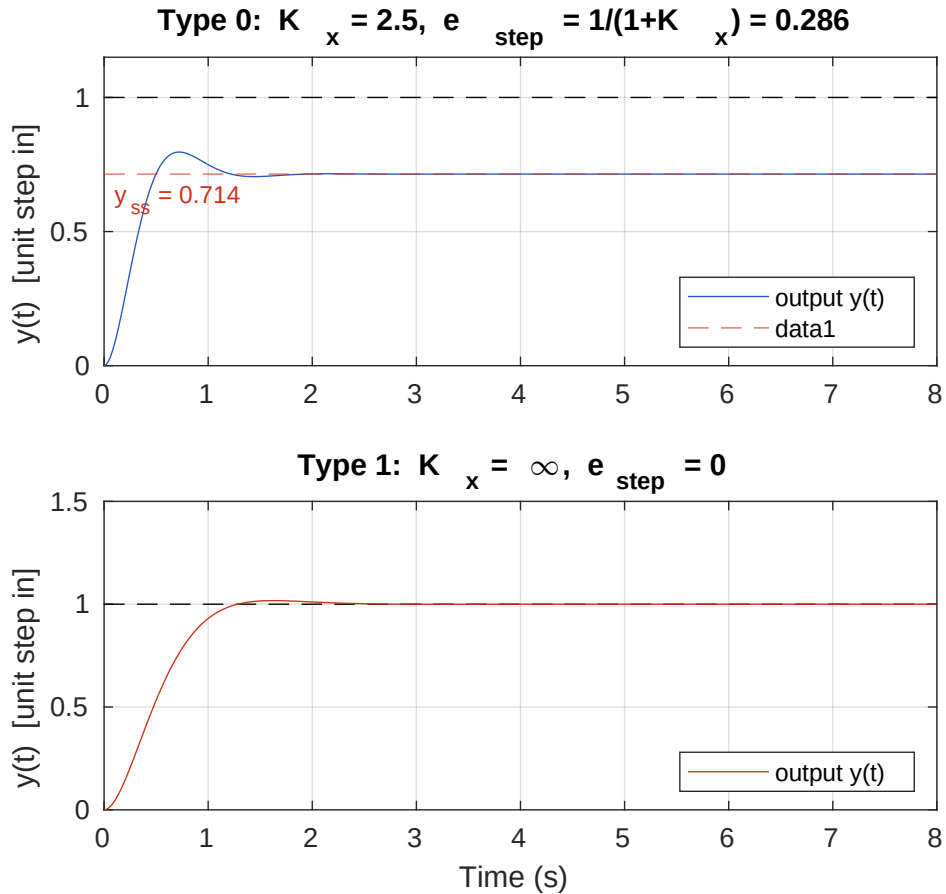


Figure 1.3: Step response comparison. *Top*: Type 0 system ($G_0 = 20/((s+2)(s+4))$, $K_x = 2.5$) settles with a finite error below the reference. *Bottom*: Type 1 system ($G_1 = 10/(s(s+5))$) tracks the step exactly.

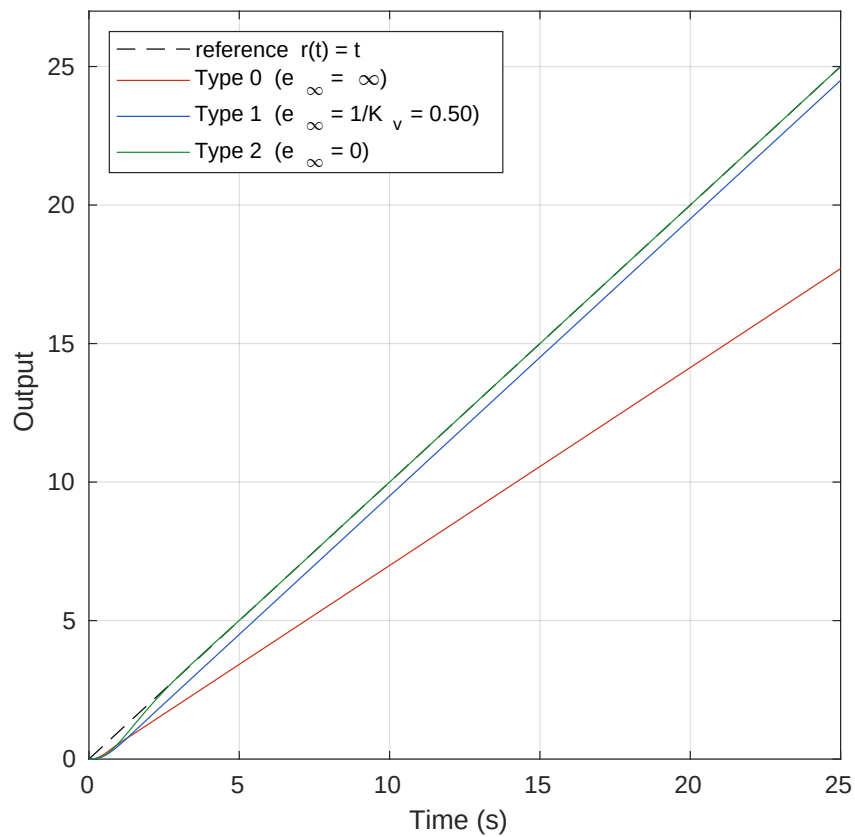


Figure 1.4: Ramp responses for three system types, all with unity feedback. Type 0 diverges (output increasingly lags behind). Type 1 converges to a constant lag of $1/K_v$. Type 2 converges to the ramp with zero steady-state error.

1.7 Design: Choosing Gain to Meet an Error Specification

In many cases the system type is set by the physics and controller architecture, leaving a scalar forward gain K as the adjustable parameter. An error specification translates directly into a requirement on K .

1.7.1 Procedure

1. Identify the system type from $G(s)$.
2. Write the relevant error constant (K_x , K_v , or K_a) in terms of K by evaluating the limit.
3. Solve for the K that satisfies the error requirement.

1.7.2 Example

Problem. The open-loop transfer function of a position control system is:

$$G(s) = \frac{K(s+3)}{s(s+6)(s+10)}$$

Find K such that the steady-state ramp error is 5 %.

Step 1: System type. One factor of s in the denominator $\Rightarrow$ Type 1. Ramp tracking yields a finite error.

Step 2: Velocity constant.

$$K_v = \lim_{s \rightarrow 0} s G(s) = \lim_{s \rightarrow 0} \frac{K s(s+3)}{s(s+6)(s+10)} = \frac{3K}{60} = \frac{K}{20}$$

Step 3: Solve for K . A 5 % ramp error requires $e_{\text{ramp}} = 1/K_v = 0.05$, so $K_v = 20$:

$$\frac{K}{20} = 20 \implies \boxed{K = 400}$$

1.8 Eliminating Steady-State Error with an Integrator

Adding integral action to the controller is the canonical way to eliminate steady-state error. The system type analysis in Section 1.5 tells us *that* it works (the type goes up by one); this section explains *why* it works at a physical level.

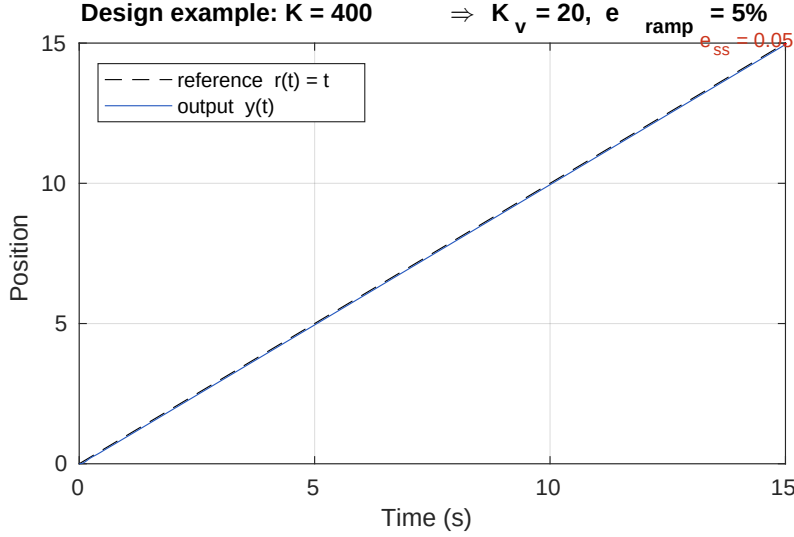


Figure 1.5: Ramp response for the design example ($K = 400$). After transients settle, the output runs parallel to the reference with a constant offset of 0.05, confirming the $e_{\text{ramp}} = 5\%$ specification for a unit-slope ramp.

1.8.1 The Adaptation Argument

The integrator contribution to the control signal is:

$$u_I(t) = K_I \int_0^t e(\tau) d\tau$$

What must be true at steady state in a *stable* closed loop?

- At steady state every signal is constant: $\dot{y} = 0$, $\dot{e} = 0$, $\dot{u} = 0$.
- The derivative of the integrator output is $\dot{u}_I = K_I e(t)$.
- For u_I to be constant at steady state, $\dot{u}_I = 0$, which requires $e(t) = 0$.

Integration enables the controller to *adapt* to persistent errors. If the output is at all below the setpoint, $e > 0$, the integrator output grows, along with the input to the plant. It stops growing only when $e = 0$ — regardless of unmodeled drag, fuel consumption changing the vehicle's mass, wind, or current opposing the vessel. The integrator compensates automatically for any source of steady-state offset.

1.8.2 Effect on System Type

Figure 1.6 shows the PI controller structure. The error $E(s)$ drives both paths: the proportional path $K_P E(s)$ provides immediate response, and the integrator path $K_I E(s)/s$ provides the accumulated bias. Combining, the open-loop transfer function becomes:

$$G(s) = \underbrace{\frac{K_P s + K_I}{s}}_{C(s)} \cdot P(s)$$

The factor of s in the denominator is the integrator — it increments the system type by one relative to $K_P \cdot P(s)$ alone.

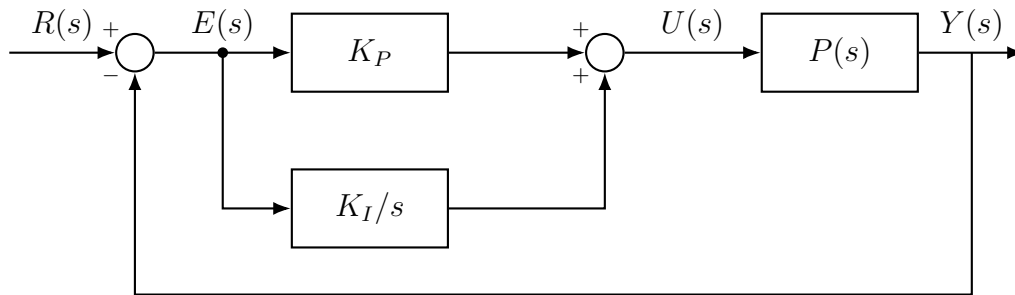


Figure 1.6: PI controller. The error $E(s)$ drives both the proportional path (instantaneous response) and the integrator path (accumulated history). The integrator output grows as long as $e \neq 0$, forcing $e \rightarrow 0$ at steady state.

1.8.3 The Trade-offs

Integral action is not free. The costs (discussed at length in the PID article) bear repeating here in the context of steady-state error:

Slower settling or more overshoot. Adding an integrator raises the closed-loop order.

A first-order plant with P control gives a first-order closed loop; with PI it becomes second-order and can oscillate. The gain K_I must be chosen carefully.

Integral windup. When the actuator saturates, the plant no longer follows the controller.

The integrator keeps charging, accumulating a large bias; when saturation clears, that bias drives overshoot. Anti-windup is required in any real implementation.

Harder to stabilize. Every integrator makes the system harder to keep stable — we will return to this in the frequency response article.

1.9 Integrator vs. Feedforward: Practical Trade-offs

There are two fundamentally different strategies for reducing steady-state error. The PID article introduced both; we now compare them directly in the framework of steady-state error analysis.

1.9.1 Feedforward Does Not Change System Type

A feedforward path injects the reference $R(s)$ directly into the plant command, bypassing the error signal. Figure 1.8 shows the architecture: the gain $1/K$ (the inverse of the plant's DC gain) routes the reference directly to the plant input, providing the known steady-state effort immediately.

At steady state, the feedforward injects r/K directly. Since the plant's DC gain is K , the output becomes $K \cdot r/K = r$ — tracking is exact. But this relies on knowing K precisely. If

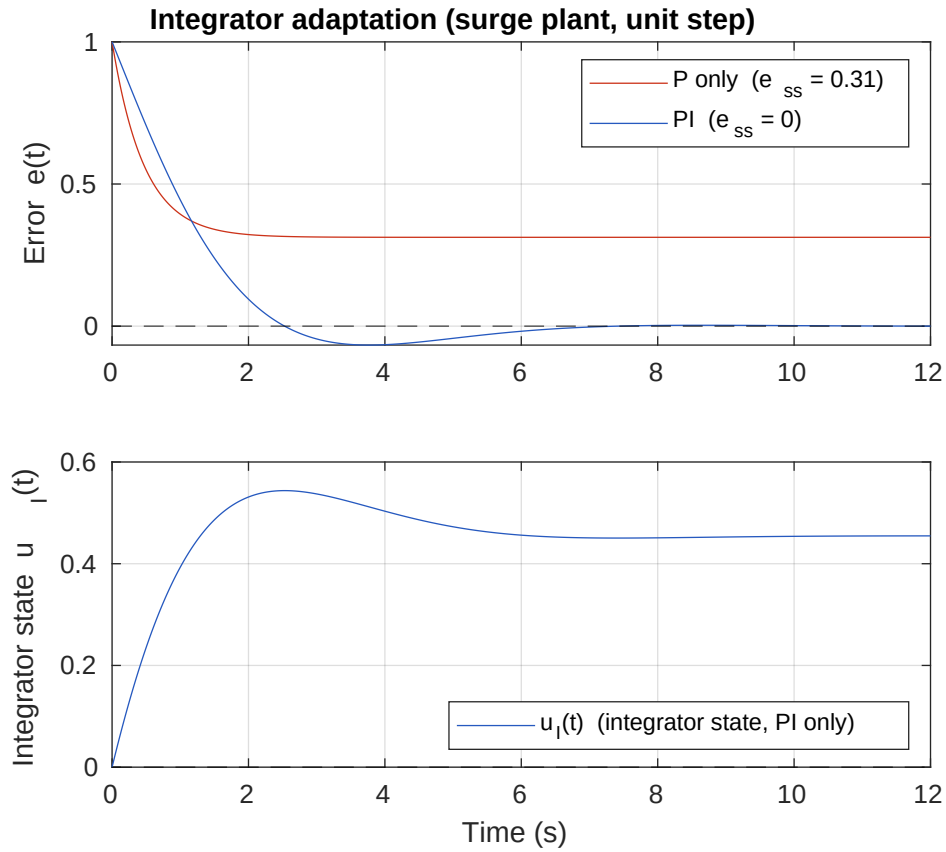


Figure 1.7: Integrator adaptation for a unit step command on the surge plant ($K = 2.2$, $\tau = 1.5$ s). *Top*: tracking error $e(t)$. P-only settles to a permanent offset; PI drives $e \rightarrow 0$. *Bottom*: integrator state $u_I(t) = K_I \int_0^t e d\tau$ (PI controller only). The integrator charges up while $e > 0$, providing a persistent bias to the plant. Once $e \approx 0$, the integrator holds constant — encoding the steady-state throttle needed to maintain the commanded speed despite the plant’s finite DC gain.

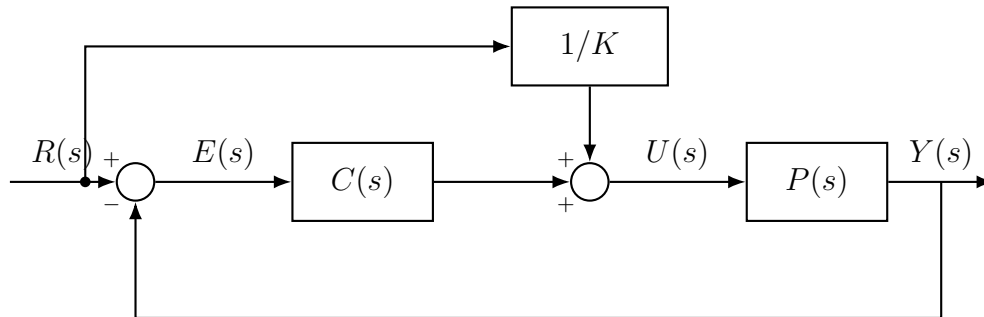


Figure 1.8: Proportional controller with feedforward gain $1/K$. The reference drives two paths: the standard error-based feedback path, and a direct path to the plant input that supplies the steady-state effort without waiting for error to accumulate.

the actual plant gain differs from the model (e.g., due to loading, temperature, or aging), a residual error remains.

More importantly for our analysis: **the feedforward path does not change the open-loop transfer function $G(s)$** . System type is unchanged. The standard error formula Eq. (1.1) does not capture feedforward because the formula models the feedback loop in isolation. The feedforward result is not in contradiction — it just requires a separate accounting.

1.9.2 Side-by-Side Comparison

Table 1.3: Integrator vs. feedforward for eliminating steady-state error.

	Integral action	Feedforward
Mechanism	Accumulates error until $e = 0$; model-free.	Directly commands the steady-state effort; model-dependent.
Model knowledge	None required; works despite gain drift or calibration error.	Requires accurate plant DC gain K ; errors leave a residual offset.
Transient response	Slower settling; possible overshoot; windup during saturation.	Fast settling; shape nearly identical to proportional-only response.
System type	Increments by 1; captured by Eq. (1.1).	No change; feedforward is outside the formula's scope.
Disturbance rejection	Yes — integrator responds to any persistent mismatch, including loads and currents.	No — feedforward acts only on the reference, not unmeasured disturbances.

1.9.3 The Combined Approach: PI with Feedforward

In practice the two strategies complement each other. Feedforward provides fast, low-overshoot response; the integrator provides robustness to model uncertainty and disturbances. Figure 1.9 compares four architectures on the surge plant.

The PI + feedforward combination is often the first choice when a plant model is available. The feedforward handles the bulk of the effort immediately; the integrator corrects whatever residual error remains due to disturbances or model mismatch. Together they provide what neither alone achieves: speed *and* robustness.

The script `sse_companion.m` (same directory as this article) generates all figures and demonstrates the numerical computations shown here.

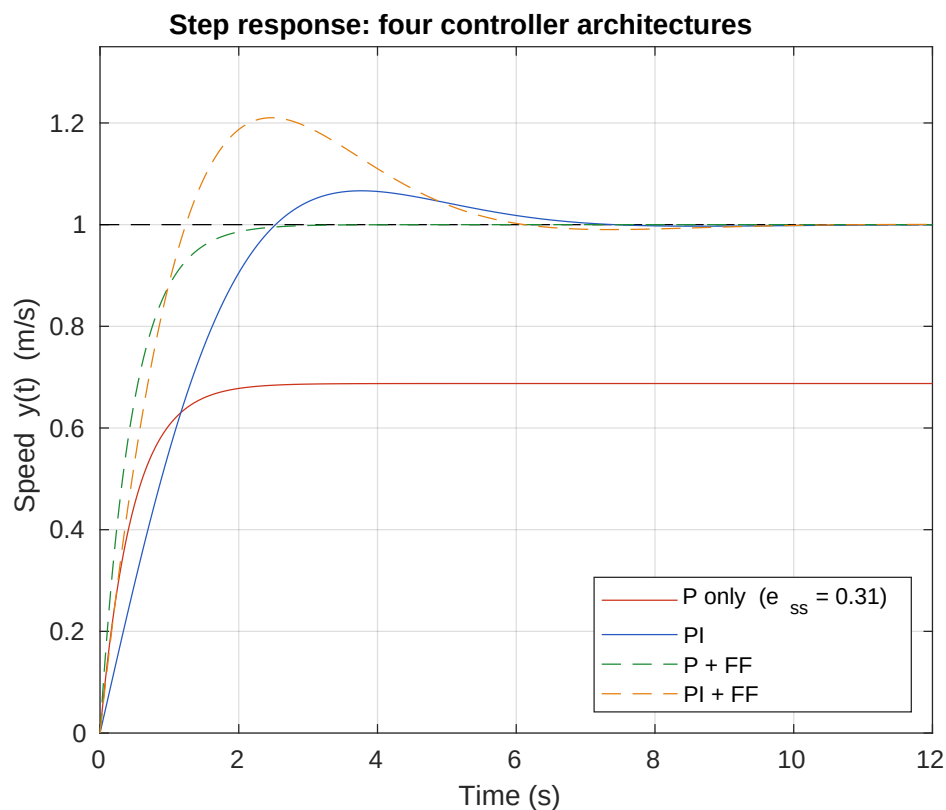


Figure 1.9: Step responses for four controller architectures on the surge plant ($K = 2.2$, $\tau = 1.5$ s). **P only:** fast but settles below the setpoint (Type 0, finite error). **PI:** zero steady-state error but slower with mild overshoot. **P + FF:** zero error *and* fast settling, but would leave a residual offset if the plant gain were mis-estimated. **PI + FF:** zero error, fast settling, and robust to model error — the preferred combination for step tracking on a modeled plant.